



Web Scraping: Introduction & Practical Using Python Flask

NIELIT Chandigarh/Ropar



"Big Data is at the foundation of all the megatrends that are happening today." — Chris Lynch



Web Scrapping



Introduction:

- Web scrapping involves extracting data from websites for further processing and analysis.
- Often used for gathering data that isn't readily available through APIs.

Key Tools:

- Python libraries such as BeautifulSoup and Scrapy.
- Automation tools like Selenium for dynamic content scraping.



What is Web Scraping?



- **Definition:**
 - The process of extracting data from websites.
- **Purpose:**
 - Automate the collection of information.
 - Convert unstructured data into a structured format.
- **Use Cases:**
 - Price comparison
 - Data analysis
 - Market research
 - News aggregation



Key Components of Web Scraping



- **HTTP Requests:**
 - Sending GET/POST requests to web servers.
- **HTML Parsing:**
 - Extracting specific elements from the webpage.
- **Libraries:**
 - Python tools like **BeautifulSoup**, **Scrapy**, and **Requests**.
- **Output Formats:**
 - CSV, JSON, Databases, etc.

Ethics and Legal Aspects



- **Guidelines:**

- Check website's Terms of Service.
- Avoid scraping sensitive or copyrighted content.
- Use polite scraping methods (e.g., rate limiting).

- **Tools to Avoid Detection:**

- User-agent rotation.
- Proxy servers.

Why Python for Web Scrapping?



- **Ease of Use:**
 - Simple syntax.
- **Libraries:**
 - Requests: For sending HTTP requests.
 - BeautifulSoup: For parsing HTML.
 - Selenium: For handling JavaScript-heavy websites.
- **Community Support:**
 - Extensive documentation and active forums.



Introduction to Flask

- **What is Flask?**
 - A lightweight web framework in Python.
- **Why Use Flask in Web Scraping?**
 - Build APIs to serve scraped data.
 - Create dashboards for data visualization.
 - Automate scraping tasks via web forms.

Workflow of Web Scrapping



1. Identify the Target Website:

1. URL and specific data requirements.

2. Send HTTP Requests:

1. Use `requests.get()` to fetch the webpage.

3. Parse HTML Content:

1. Extract elements with BeautifulSoup.

4. Store Data:

1. Save in a database or export as CSV/JSON.

5. Build API with Flask:

1. Serve scraped data dynamically.

Hands-On Setup



- **Install Required Libraries:**
 - pip install flask beautifulsoup4 requests
- **Basic Project Structure:**
 - app.py (Flask app)
 - scraper.py (Scraping logic)
 - templates/ (HTML files)

Code Walkthrough: Scraping Logic



- **Example: Scraping Titles from a Blog**

```
import requests  
from bs4 import BeautifulSoup  
def scrape_blog():  
    url = https://example-blog.com  
    response = requests.get(url)  
    soup = BeautifulSoup(response.text, 'html.parser')  
    titles = [title.text for title in soup.find_all('h2')]  
    return titles
```

Code Walkthrough: Flask Integration



- Example:

```
from flask import Flask, jsonify
from scraper import scrape_blog
app = Flask(__name__)
@app.route('/')
def home():
    data = scrape_blog()
    return jsonify(data)
if __name__ == '__main__':
    app.run(debug=True)
```

Demo Time



- **Steps:**

- Run the Flask app: `python app.py`
- Open the browser:
`http://127.0.0.1:5000/`
- View the scraped data in JSON format.

- **Visualization Ideas:**

- Display the data in a table or graph.

Challenges in Web Scraping

- **Dynamic Content:**
 - JavaScript-heavy websites.
- **Anti-Scraping Mechanisms:**
 - CAPTCHAs, IP blocking.
- **Changing Website Structure:**
 - Frequent updates in HTML layout.



Best Practices

- Use proper headers (e.g., User-Agent).
- Respect robots.txt.
- Cache frequently scraped data.
- Handle errors and timeouts.



Q&A



- **Let's Discuss:**
- Any questions about setup or implementation?
- Real-world use cases for your projects.



Conclusion



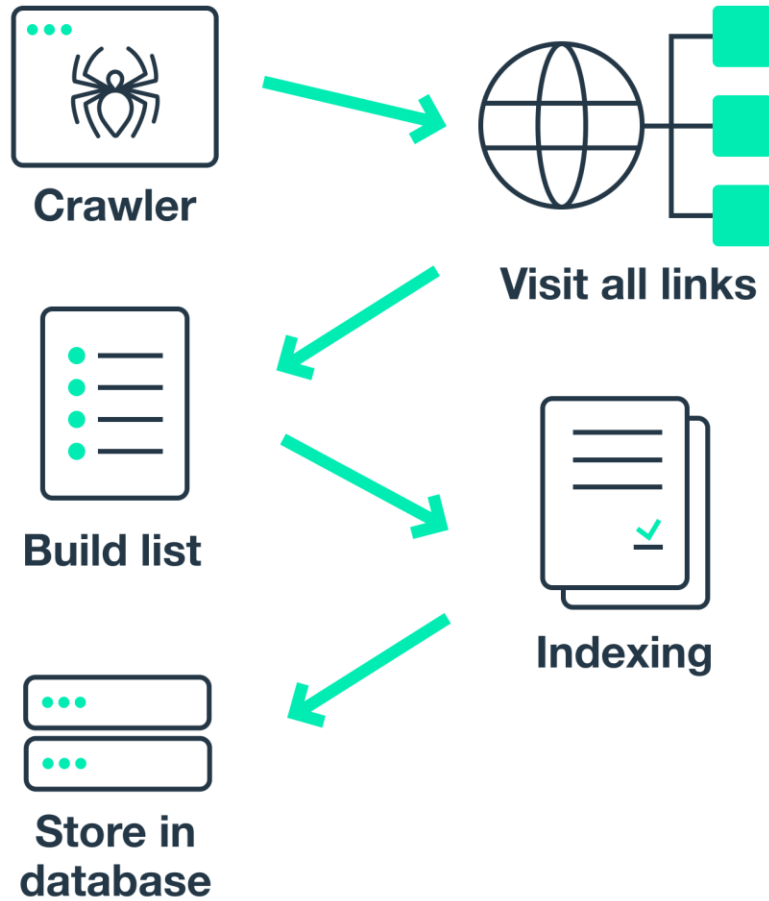
- **Key Takeaways:**

- Python is a powerful tool for web scraping.
- Flask makes it easy to build APIs and serve data.
- Ethical scraping is critical for sustainable use.

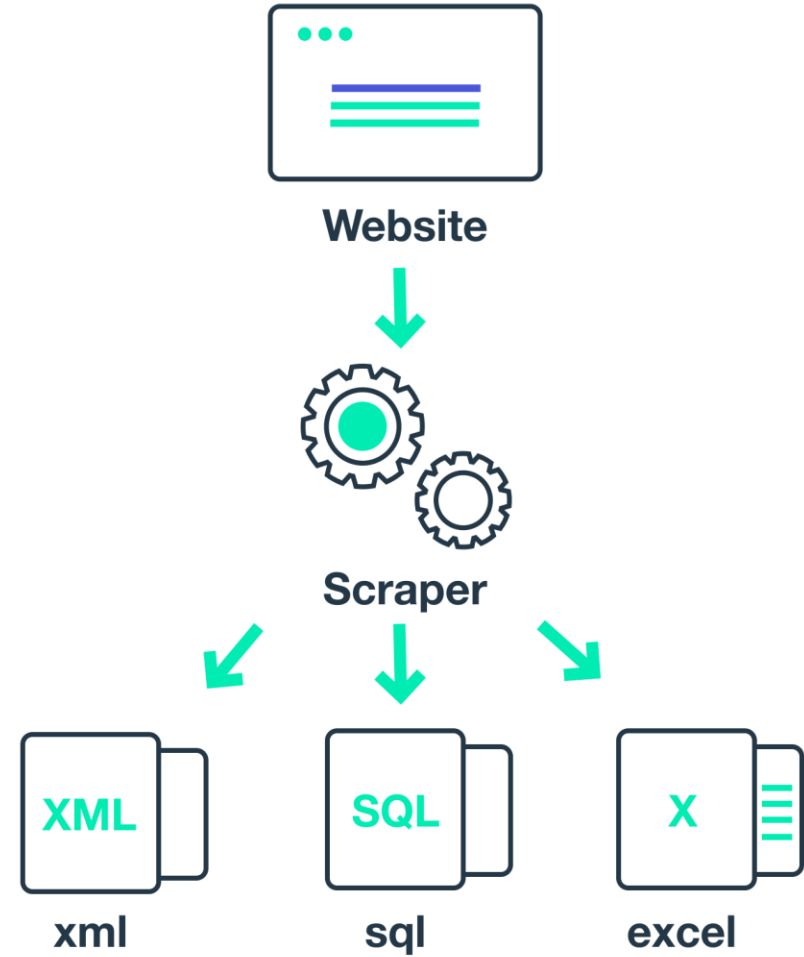
- **Next Steps:**

- Explore advanced topics: Selenium, Scrapy.
- Implement a complete data pipeline.

Web Crawler



Web Scrapping





app.py



main ▾ webscrapingexample / app.py

LovnishVerma Upload 4 files 65e962b VERIFIED

less than a minute ago

</> raw Copy download link history blame edit delete Safe

875 Bytes

```
1 from flask import Flask, render_template, request
2 import requests
3 from bs4 import BeautifulSoup
4
5 app = Flask(__name__)
6
7 @app.route("/")
8 def index():
9     return render_template("index.html")
10
11 @app.route("/scrape", methods=["POST"])
12 def scrape():
13     url, tag = request.form.get("url"), request.form.get("tag")
14     if not url or not tag:
15         return render_template("result.html", error="Both URL and Tag are required.")
16
17     response = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
18     response.raise_for_status() # This will automatically raise an error if the request fails
19
20     soup = BeautifulSoup(response.text, "html.parser")
21     elements = [e.get_text() for e in soup.find_all(tag)]
22
23     return render_template("result.html", tag=tag, url=url, title=soup.title.string or "No Title", elements=elements)
24
25 if __name__ == "__main__":
26     app.run(debug=True)
```



templates/index.html



webscrapingexample/ templates/index.html

Edit	Preview
<pre>1 <html> 2 <head> 3 <!-- Favicons --> 4 <link rel="icon" type="image/png" sizes="48x48" href="https://lovnishverma.in/favicon-48.png"> 5 <link rel="icon" type="image/png" sizes="192x192" href="https://lovnishverma.in/android-chrome-192x192.png"> 6 <link rel="apple-touch-icon" href="https://lovnishverma.in/apple-touch-icon.png"> 7 <link rel="shortcut icon" href="https://lovnishverma.in/favicon.ico"> 8 <title>Web Scraper</title> 9 <style> 10 body { 11 font-family: Arial, sans-serif; 12 margin: 0; 13 padding: 0; 14 background-color: #f9f9f9; 15 text-align: center; 16 } 17 .logo { 18 margin-top: 30px; 19 width: 128px; 20 height: 128px; 21 } 22 </style> 23 </head> 24 <body> 25 27 <h1>Customizable Web Scraper</h1> 28 <form action="/scrape" method="POST"> 29 <label for="url">Enter URL:</label>
 30 <input type="text" id="url" name="url" placeholder="https://example.com" required>

 31 32 <label for="tag">Enter Tag to Scrape (e.g., p, h1, img):</label>
 33 <input type="text" id="tag" name="tag" placeholder="p" required>

 34 35 <button type="submit">Scrape</button> 36 </form> 37 38 </body> 39 </html></pre>	



templates/result.html



main ▾ [webscrapingexample](#) / templates / result.html

LovnishVerma Update templates/result.html 089de8d VERIFIED

`</>` raw Copy download link history blame edit delete Safe

```
1 <html>
2 <head>
3     <!-- Favicons -->
4     <link rel="icon" type="image/png" sizes="48x48" href="https://lovnishverma.in/favicon-48.png">
5     <link rel="icon" type="image/png" sizes="192x192" href="https://lovnishverma.in/android-chrome-192x192.png">
6     <link rel="apple-touch-icon" href="https://lovnishverma.in/apple-touch-icon.png">
7     <link rel="shortcut icon" href="https://lovnishverma.in/favicon.ico">
8     <title>Scraped Results</title>
9 </head>
10 <body>
11     <h1>Scraped Results</h1>
12     {% if error %}
13         <p style="color: red;">{{ error }}</p>
14     {% else %}
15         <h2>Title: {{ title }}</h2>
16         <h3>Content from: <a href="{{ url }}" target="_blank">{{ url }}</a></h3>
17         <h3>Scraped Elements for Tag: &lt;{{ tag }}&gt;</h3>
18         <ul>
19             {% for element in elements %}
20                 <li>{{ element }}</li>
21             {% else %}
22                 <li>No content found for tag &lt;{{ tag }}&gt;.</li>
23             {% endfor %}
24         </ul>
25     {% endif %}
26     <br>
27     <a href="/">Go Back</a>
28 </body>
29 </html>
30
```

Title: Government of India : National Institute of Electronics & Information Technology

Content from: <https://nielit.gov.in/chandigarh/tender>

Scraped Elements for Tag: <p>

- राष्ट्रीय इलेक्ट्रॉनिकी एवं सूचना प्रौद्योगिकी संस्थान ,चंडीगढ़
- National Institute of Electronics & Information Technology, Chandigarh
-
- The last date for submission of technical & financial bid of Tender No: NIELIT/CH/PUR/232/V-4/2024/01 for Hiring of vendor for Videography and Photography services during Examination has been extended upto 16-12-2024 5:00 P.M.
- Limited Tender Document for Videography and Photography services for Examination
- EOI for “Employability Enhancement & Livelihood Training Program [EELTP] of SC/ST & EWS (Women) Youth through Capacity Building and Skill Development in IECT”
-
- EOI for “Employability Enhancement & Livelihood Training Program [EELTP] of SC/ST & EWS (Women) Youth through Capacity Building and Skill Development in IECT”
-
- Tender for Conduct of Physical Efficiency Test & Physical Measurement Test
- EOI for Training Partners for the Aspirational District Project at Moga and Ferozpur
-
- GeM Bid for Hiring of Security Vehicles for Transport of Confidential Material
- Tender for Books
- GeM-Bid for Frisking during Examinations
- GeM bid for Solar Street Lights System

➤ **HF SPACE:** <https://huggingface.co/spaces/LovnishVerma/web scrapingexample>
GitHub Repo: https://github.com/lovnishverma/web_scraping
LIVE DEMO: <https://lovnishverma-web scrapingexample.hf.space/>